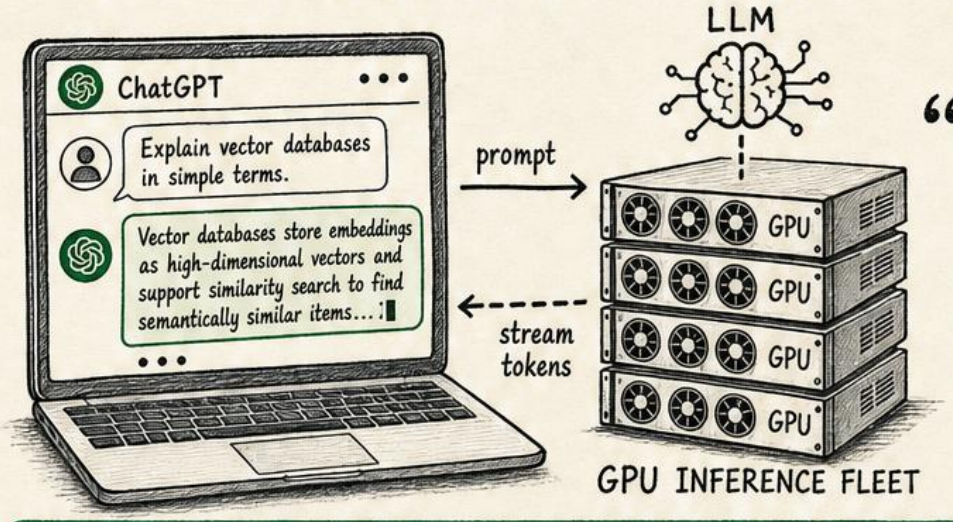
@ codedsoul_05

LLM SD HANDBOOK v1.0 ★

CHATGPT SYSTEM DESIGN

— HANDWRITTEN HANDBOOK —

Designing a Large-Scale LLM Chat & Inference Platform ☺



"First Token Fast. Every Token Cheap. Every Answer Safe."

Low TTFT • Throughput • GPU Efficiency • Safety • Cost

What's Inside ?

1. Requirements & Scale
2. High-Level Architecture
3. APIs & Streaming
4. Conversation Storage
5. Inference Fundamentals
6. Batching & Scheduling
7. GPU Fleet & Autoscaling
8. Context Management
9. RAG & Tool Calling
10. Caching & Cost Control

+ Safety, Reliability & Interview Q&A

Key Numbers & Terms

- TTFT — time to first token
- TPOT — time per output token
- Prefill — process the prompt (compute-bound)
- Decode — one token at a time (memory-bound)
- KV Cache — grows with context length
- Batching — continuous, not static
- Unit of cost — the token

Latency is felt per token, not per request.

Topics Covered

- ☑ Streaming (SSE)
- ☑ Token Economics
- ☑ KV Cache & Paging
- ☑ Continuous Batching
- ☑ Tensor Parallelism
- ☑ Quantization
- ☑ Speculative Decoding
- ☑ Vector Search / RAG
- ☑ Semantic Caching
- ☑ Rate Limits & Quotas
- ☑ Moderation & Safety
- ☑ Observability

Who is this for ?

- ★ System Design Interview Preparation
- ★ Backend / ML Platform Engineers
- ★ Anyone building on top of LLMs! ☺

"Serving a model is not one big request — it is millions of tiny token deadlines."

Stream Early. Batch Aggressively. Cache Everything You Can. ★

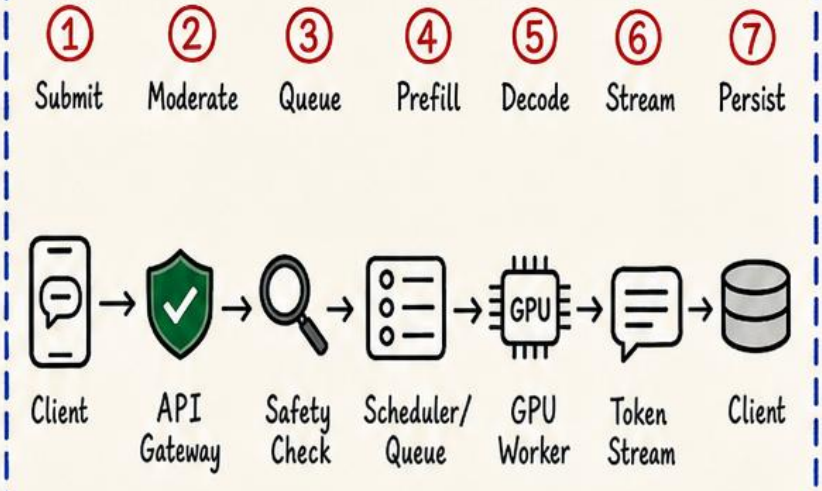
01 — REQUIREMENTS & SCOPE

Serve conversational AI to millions — fast first token, sustained throughput, bounded cost.

CORE FEATURES

- Send a prompt, receive a streamed reply
- Multi-turn conversations with memory
- Conversation history, rename, delete
- Regenerate and edit a message
- File and image attachments
- Tool / function calling
- Multiple models and tiers





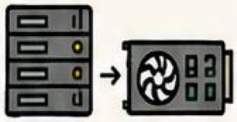
FUNCTIONAL REQUIREMENTS

1. Chat



multi-turn, streamed, stateful conversations

2. Inference



batched GPU execution across many users

3. Context



assemble history within the model's window

4. Safety



screen input and output before delivery

NON-FUNCTIONAL REQUIREMENTS

- Low TTFT — first token in under ~1 second
- Steady TPOT — tokens faster than reading speed (~20-40/s)
- Scalability — millions of concurrent sessions
- GPU efficiency — utilisation drives cost per token
- Availability — degrade to a smaller model, never a blank page
- Safety & privacy — moderation, no training on private data by default

OUT OF SCOPE

- ✗ Model training and fine-tuning pipelines
- ✗ Transformer mathematics
- ✗ Billing and subscription internals

INTERVIEW TIP

Open by separating the two systems: a normal stateless web app, and a GPU scheduling problem. Almost all the difficulty is in the second.

A chat product on the outside. A scheduler on the inside.

02 — CAPACITY ESTIMATION

In LLM systems the unit of capacity is the TOKEN, not the request.

ASSUMPTIONS

Daily active users	Conversations / user / day	Messages per conversation	Input context	Output	Peak factor
= 100 Million	= 3	= 10	≈ 1,000 tokens	≈ 300 tokens	= 3 × average

Illustrative estimates

REQUEST TRAFFIC

Messages / day = 100M × 3 × 10
= 3 Billion

Average = 3B ÷ 86,400
≈ 35,000 requests/sec

Peak = 35K × 3
≈ 105,000 requests/sec

≈ 100K concurrent generations at peak

TOKEN THROUGHPUT (the real metric)

Output tokens/day = 3B × 300
= 900 Billion

Average output = 900B ÷ 86,400
≈ 10.4 Million tokens/sec

Output tokens dominate cost — decode is the expensive phase.

GPU FLEET SIZING

Assume one GPU sustains ≈ 2,000 output tokens/sec when well batched

GPUs needed = 10.4M ÷ 2,000
≈ 5,200 GPUs

At peak (×3) ≈ 15,600 GPUs

Plus redundancy and multi-region headroom

Throughput per GPU depends on model size, batch size and quantization.

KV CACHE MEMORY

KV per token ≈ 2 × layers × heads × head_dim × bytes

Worked example ≈ 0.5 MB per token for a large model

Sequence of 2,000 tokens ≈ 1 GB per active request

An 80 GB GPU therefore holds only tens of concurrent long sessions.

KV cache — not FLOPs — usually limits batch size.

STORAGE

Messages/day = 3B × ~1 KB
≈ 3 TB/day

With metadata and replication
≈ 9 TB/day

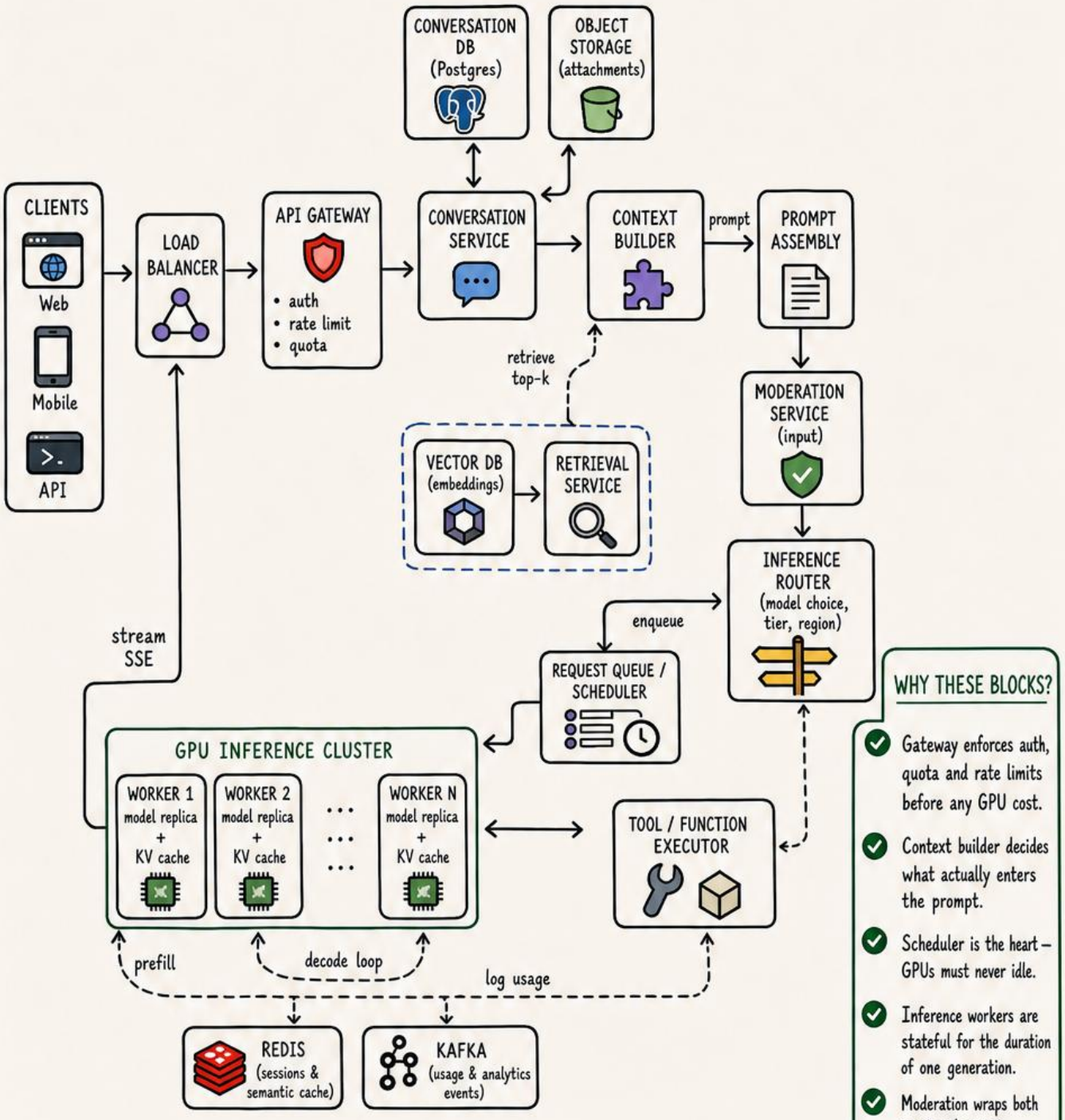
Text is cheap; GPUs are not.

WHAT THESE NUMBERS TELL US

Token-bound → batch continuously → KV-cache-bound → page and evict cache → Expensive GPUs → queue, prioritise, autoscale → Cheap text → store full history in a normal database

Count tokens, then count GPUs. Everything else follows.

03 — HIGH-LEVEL ARCHITECTURE



WHY THESE BLOCKS?

- Gateway enforces auth, quota and rate limits before any GPU cost.
- Context builder decides what actually enters the prompt.
- Scheduler is the heart — GPUs must never idle.
- Inference workers are stateful for the duration of one generation.
- Moderation wraps both input and output.

REQUEST FLOW

1. Authenticate
2. Moderate input
3. Build context
4. Route model
5. Queue
5.
6. Prefill
7. Decode & stream
8. Moderate output
9. Persist & meter

KEY DESIGN

- Web tier is stateless; GPU tier is scheduled, not load-balanced naively.
- Never send a request to a GPU before quota and safety checks.
- Separate control plane (fast, cheap) from data plane (slow, expensive).

INTERVIEW TIP

Draw two planes: the cheap stateless request path and the expensive GPU execution path.

Guard the GPU — it is the scarcest resource in the system. ★

04 — APIs & STREAMING

REST to manage conversations. Server-Sent Events to stream tokens.

CONVERSATION APIs (REST)

- POST /v1/conversations
  create a conversation
- GET /v1/conversations?cursor=
  list
- GET /v1/conversations/{id}/
  messages?cursor=
- PATCH /v1/conversations/{id}
  rename, archive
- DELETE /v1/conversations/{id}

COMPLETION API

- POST /v1/chat/completions

```
{
  "model": "gpt-x",
  "conversation_id": "conv_88",
  "messages": [
   { "role": "user", "content": "Explain CAP" }
  ],
  "stream": true,
  "max_tokens": 500,
  "temperature": 0.7,
  "idempotency_key": "req_9f21"
}
```

SSE TOKEN STREAM

```
data: {"delta":"CAP"}
data: {"delta":" theorem"}
data: {"delta":" says"}
data: {"finish_reason":"stop",
       "usage":{"in":812,"out":143}}
data: [DONE]
```

Note: Each chunk is one or a few tokens. The connection stays open for the whole generation.

WHY SSE, NOT WEBSOCKET

- One-way server → client is all we need
- Plain HTTP, works through proxies and CDNs
- Auto-reconnect built into the browser
- WebSocket only when you need duplex (voice, live editing)

OTHER CONTRACT DETAILS

- ✓ Idempotency key prevents duplicate charged generations
- ✓ Return usage (input/output tokens) in the final event
- ✓ Include request_id for tracing and support
- ✓ Long timeouts, plus heartbeat comments to keep proxies alive
- ✓ Version the API — prompt formats change

CANCELLATION MATTERS

Client disconnects → the server MUST stop generating.

- Propagate cancellation to the scheduler and GPU worker
- Free the KV cache immediately
- Bill only the tokens actually produced

An abandoned stream that keeps decoding is pure wasted money.



Stream the answer; never make the user wait for the last token. ★

05 — DATA MODEL & STORAGE

Conversations are ordinary data. The hard part sits on the GPU, not in the database.

USER

user_id	PK
email_hash	
plan	
created_at	

CONVERSATION

conversation_id	PK
user_id	
title	
model	
created_at	
updated_at	

MESSAGE

conversation_id	PK
seq	CK
role	
content	
token_count	
created_at	

GENERATION

generation_id	PK
message_id	
model	
input_tokens	
output_tokens	
latency_ms	
finish_reason	

ATTACHMENT

attachment_id	PK
conversation_id	
type	
storage_path	
size	

USAGE

user_id	PK
day	CK
input_tokens	
output_tokens	
cost	

MEMORY

user_id	PK
fact_id	CK
text	
embedding_id	
updated_at	

KEY DECISIONS

- ✓ role is one of system | user | assistant | tool
- ✓ Messages are append-only and ordered by seq
- ✓ Store token_count at write time — you need it for context assembly
- ✓ An edited message forks the thread; keep both branches
- ✓ Attachments live in object storage, only the path in the DB

STORAGE CHOICES

PostgreSQL	users, conversations, messages, usage
Object Storage (S3/GCS)	files, images, exported chats
Redis	sessions, rate-limit counters, semantic cache
Vector DB (pgvector / Milvus / Pinecone)	embeddings for retrieval
Kafka	usage and telemetry events
Warehouse	analytics, evaluation, abuse review

PARTITIONING

- Shard by user_id — a conversation never spans users
- Partition messages by conversation_id + time
- Archive cold conversations to cheap storage

A single conversation can grow to thousands of messages — always paginate.

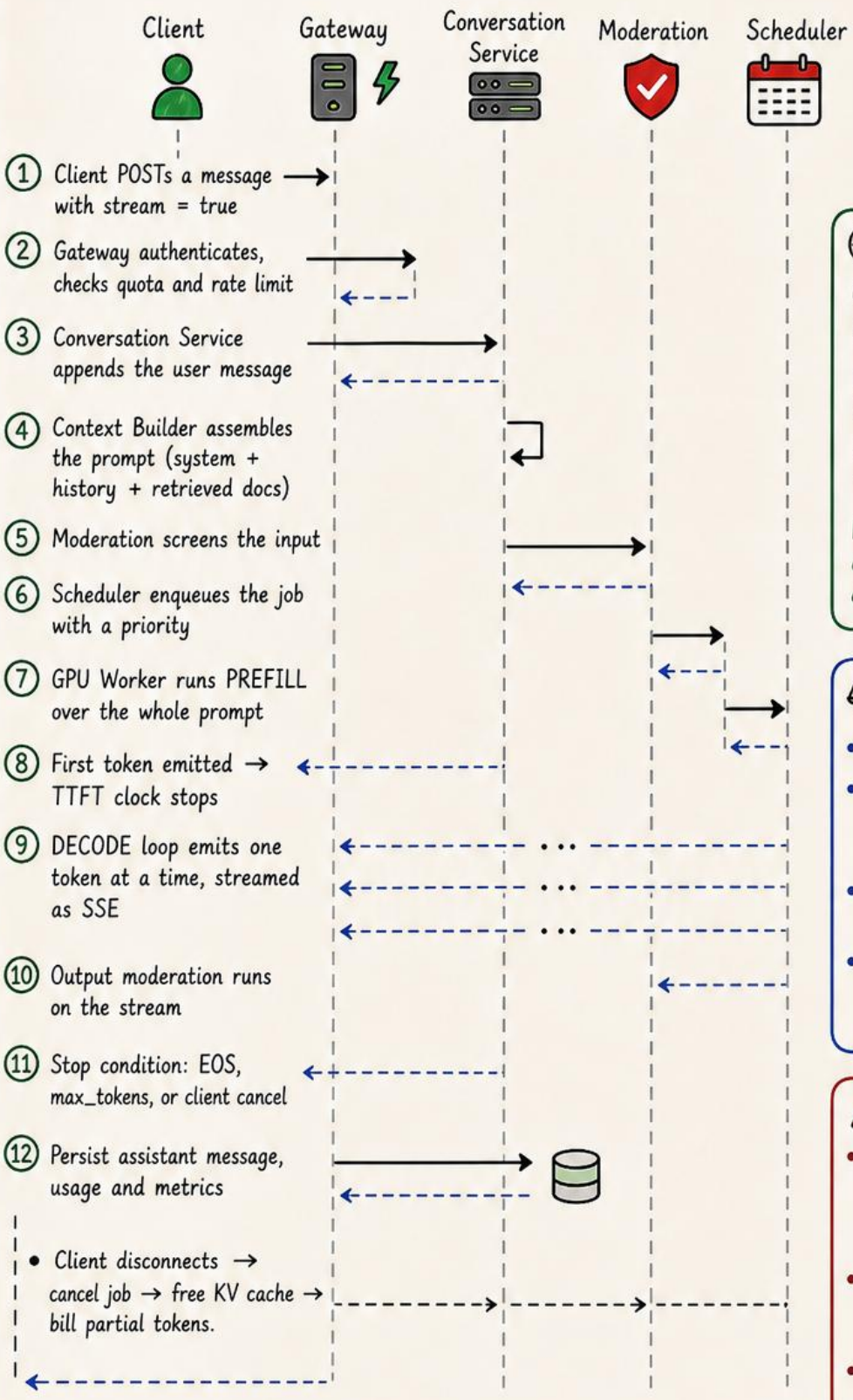
ACCESS PATTERNS (DRIVE THE DESIGN)

1. Load the sidebar: recent conversations for a user
2. Load the last N messages of one conversation
3. Append a message and its generation record
4. Sum today's tokens for quota enforcement
5. Semantic search over past conversations

Never store raw KV cache — it is huge, model-specific and worthless once the model version changes.

Text is cheap to store. Choose what to send, not what to keep. ★

06 — REQUEST LIFECYCLE



WHERE THE TIME GOES

Queue wait + Prefill = TTFT
Decode = output_tokens × TPOT

Example:
800 ms TTFT +
300 tokens × 25 ms
≈ 8.3 s total

Note: Users forgive a long answer. They do not forgive a slow first token.

PRIORITY & FAIRNESS

- Paid tier ahead of free tier
- Short prompts can be admitted alongside long ones
- Starvation guard: age raises priority over time
- Reject early with a clear message when the queue is too deep

FAILURE POINTS

- Worker crashes mid-generation → retry on another worker (prompt is idempotent)
- Queue too deep → shed load or fall back to a smaller model
- Moderation flags output → stop the stream and replace the message
- Timeout → return partial text plus an explicit error event

INTERVIEW TIP *Say where TTFT is measured — it separates people who have run inference from people who have only read about it.*

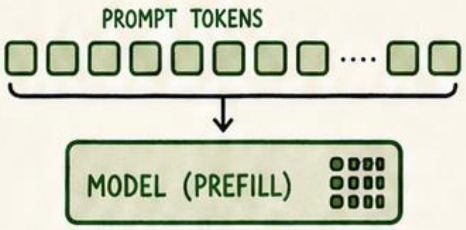
Everything before the GPU is cheap. Spend the GPU wisely.

07 — INFERENCE FUNDAMENTALS

Two very different phases share one GPU — understand both or nothing else makes sense.

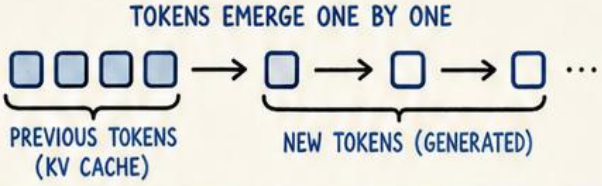
1. PREFILL (prompt processing)

- Processes ALL prompt tokens in parallel
- Compute-bound — the GPU's matrix units are saturated
- Cost grows with prompt length
- Produces the initial KV cache
- This dominates time-to-first-token



2. DECODE (generation)

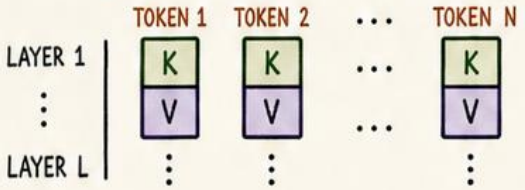
- Generates ONE token at a time, autoregressively
- Each token depends on all previous ones
- Memory-bandwidth-bound — weights are re-read every step
- Cannot be parallelised within a single sequence
- Dominates total latency for long answers



3. THE KV CACHE

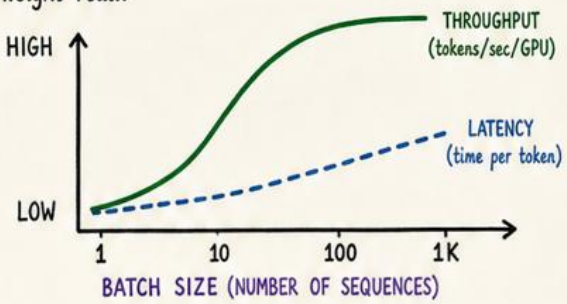
- Stores the key/value tensors of every previous token
- Avoids recomputing the whole history each step
- Size ≈ 2 × layers × heads × head_dim × seq_len × bytes
- Grows linearly with context length AND batch size

⚠ KV cache memory — not compute — usually caps how many users share a GPU.



4. WHY BATCHING WORKS

During decode the GPU is idle waiting on memory. Running many sequences together reuses the same weight read.



Batching buys throughput almost for free — until memory runs out.

5. KEY METRICS

- **TTFT** — time to first token (queue + prefill)
- **TPOT / ITL** — time per output token during decode
- **Throughput** — total output tokens per second per GPU
- **Goodput** — tokens delivered within the latency target
- GPU utilisation and KV cache occupancy

6. MAKING IT FASTER

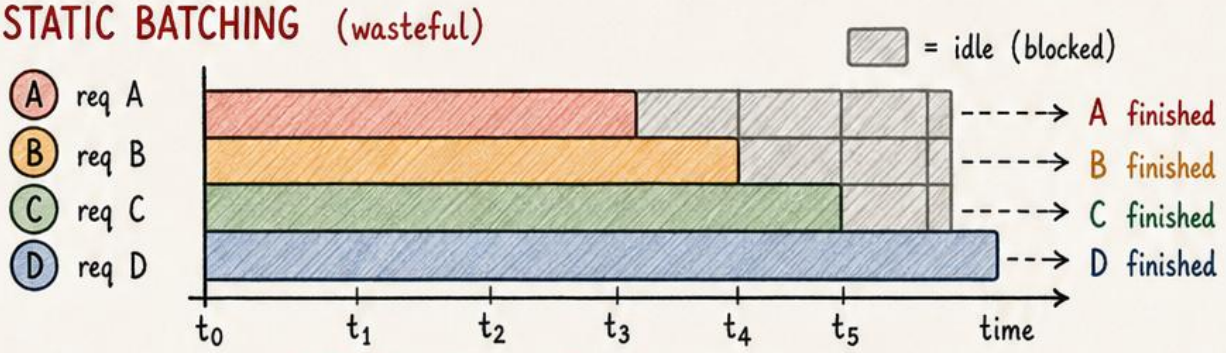
- **Quantization** (FP8 / INT8 / 4-bit) — smaller weights, faster memory reads
- **Tensor parallelism** — split one model across GPUs
- **Pipeline parallelism** — split layers across GPUs
- **Speculative decoding** — a small draft model proposes several tokens, the big model verifies them in one pass
- **FlashAttention** — memory-efficient attention kernels

Prefill is compute. Decode is memory. Optimise them separately.

08 — BATCHING & SCHEDULING

The scheduler decides your cost per token. It is the most important component you will write.

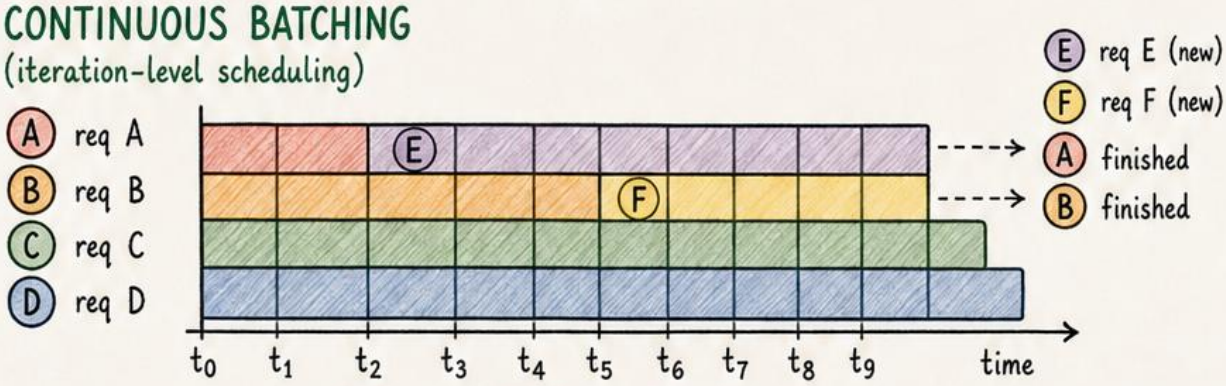
STATIC BATCHING (wasteful)



Whole batch waits for the slowest sequence.

CONTINUOUS BATCHING

(iteration-level scheduling)



Slots are refilled every decoding step.

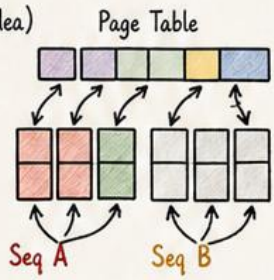
Same GPU. Two to four times the throughput.

HOW CONTINUOUS BATCHING WORKS

1. Maintain a running batch of active sequences
2. Every decode step, emit one token for each
3. Finished sequences leave the batch immediately
4. Waiting requests are admitted into the free slots
5. Repeat — the batch composition changes constantly

PAGED KV CACHE (PagedAttention idea)

- KV cache stored in fixed-size blocks, like OS memory pages
- A sequence's cache need not be contiguous
- Almost eliminates memory fragmentation
- Blocks can be SHARED between sequences with the same prefix



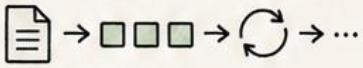
ADMISSION CONTROL

Before admitting a request, check:

- Is there enough free KV cache for its expected length?
- Will admitting it push TPOT past the latency target?
- Does its priority beat what is already queued?

Over-admitting causes preemption and cache thrash — throughput collapses.

CHUNKED PREFILL



Long prompts are split into chunks and interleaved with decode steps, so one huge prompt does not stall everyone else's token stream.

SCHEDULER GOALS

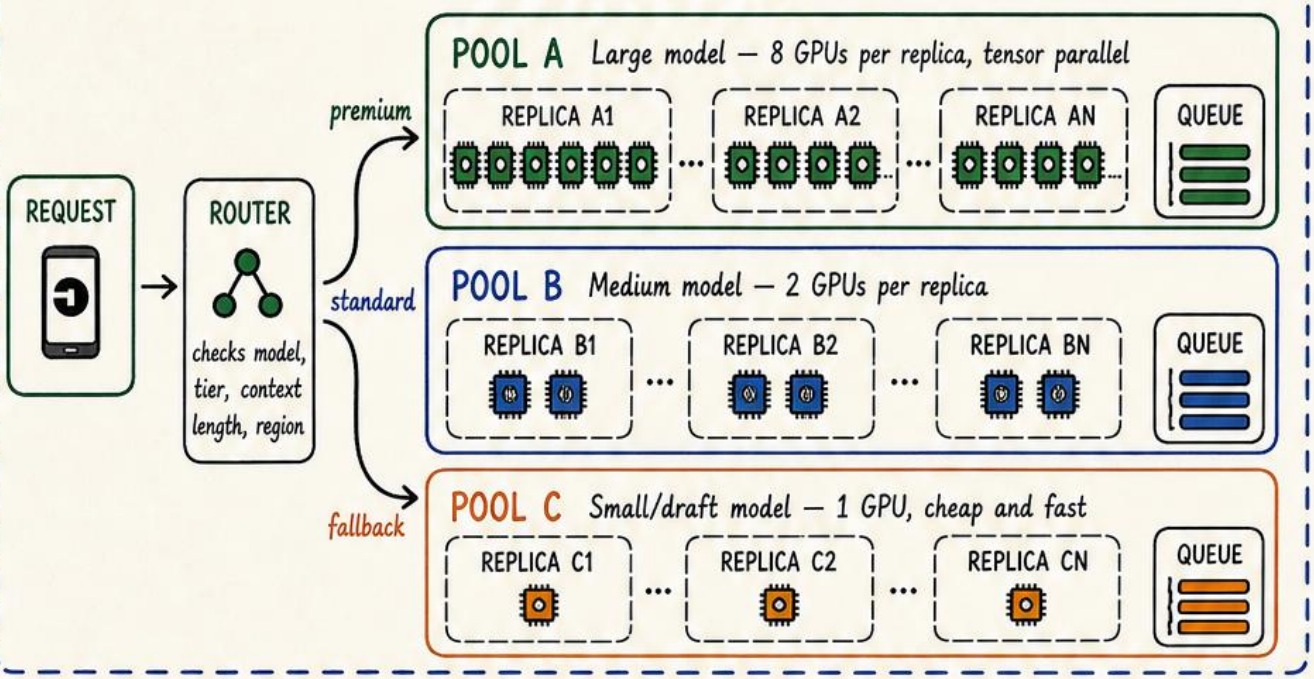
- Maximise tokens/sec
- Keep TTFT low
- Keep TPOT stable
- Respect priority
- Never idle the GPU

Fill every slot, every step. Idle GPU time is money burned.

09 — GPU FLEET & ROUTING

Many models, many tiers, one expensive fleet — routing decides who runs where.

MODEL ROUTING



1. MODEL PARALLELISM

- Tensor parallelism — split each layer's matrices across GPUs (needs fast interconnect)
- Pipeline parallelism — different layers on different GPUs
- A very large model may not fit on one GPU at all

Note: Replica = the smallest group of GPUs that can serve the model.

2. AUTOSCALING IS HARD

- Loading a large model takes minutes, not seconds
- GPU capacity is scarce and expensive
- Scale on queue depth and TTFT, not CPU

Tactics:

- keep warm standby replicas
- pre-load weights onto local NVMe
- scale down slowly
- use spot capacity only for batch jobs

3. PLACEMENT & LOCALITY

- Route follow-up turns to a replica that already holds the prefix cache
- Sticky routing by conversation_id improves cache hit rate
- Keep users in the nearest region for network latency

4. TIERING & DEGRADATION

Under heavy load, in order:

1. Queue with priority
2. Reduce max_tokens
3. Route to a smaller model
4. Disable optional features (retrieval, tools)
5. Reject politely with a retry hint

Degrade the answer, never the availability.

5. COST LEVERS

- Batch size
- Quantization
- Speculative decoding
- Prefix cache hit rate
- Right-sized model per task

A cheaper model that is good enough beats a big model on every request.

TRADE-OFF

MORE GPU PER REPLICA	MORE REPLICAS
✓ Higher quality	✓ Faster scale-out
✓ Handles longer context	✓ Better availability
✗ Slower scale-up	✓ More tenants
✗ Longer cold start	✗ Lower max context
✗ Fewer tenants per host	✗ More routing overhead
	✗ Higher total cost

★ Most LLM cost problems are routing problems in disguise.

Right model, right pool, right region — then optimise the kernel. ★

10 — CONTEXT MANAGEMENT

The model has no memory. Every turn, you rebuild its entire world from scratch.

THE CONTEXT BUDGET

System prompt	User memory	Retrieved docs	Recent history	Current message	RESERVED for output
500	300	2,000	4,000	200	1,000

Window = input + output. Always reserve room for the answer.

⚠ Forget to reserve output space and the model gets truncated mid-sentence.

WHAT GOES IN, IN PRIORITY ORDER

1. System prompt and safety rules (always)
2. The current user message (always)
3. Recent turns (most recent first)
4. Retrieved documents (top-k, trimmed)
5. Long-term user memory (facts, preferences)
6. Older turns — summarised, then dropped

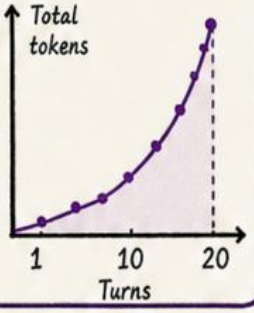
TRUNCATION STRATEGIES

- Sliding window — keep the last N turns (simple, forgets facts)
- Summarise-and-drop — compress old turns into a running summary
- Hierarchical — summary of summaries for very long chats
- Selective recall — embed old turns, retrieve only what is relevant now

✱ Never truncate from the start of the system prompt.

THE COST ANGLE

- Input tokens are billed and processed on every single turn.
- A 20-turn chat resending full history is quadratic in total tokens.
- Fixes: summarise aggressively, cache the prefix, trim retrieved documents.



PREFIX CACHING (huge win)

The system prompt and early history are identical every turn.
Cache their KV blocks and reuse them → prefill skips that work entirely.

- Requires the prefix to be byte-identical
- Keep variable content at the END of the prompt

✓ Put the stable text first. Order alone can double your cache hit rate.

LONG-TERM MEMORY

Extract durable facts ("prefers Python", "based in Pune")
Store with embeddings, retrieve only when relevant, let the user view and delete them.

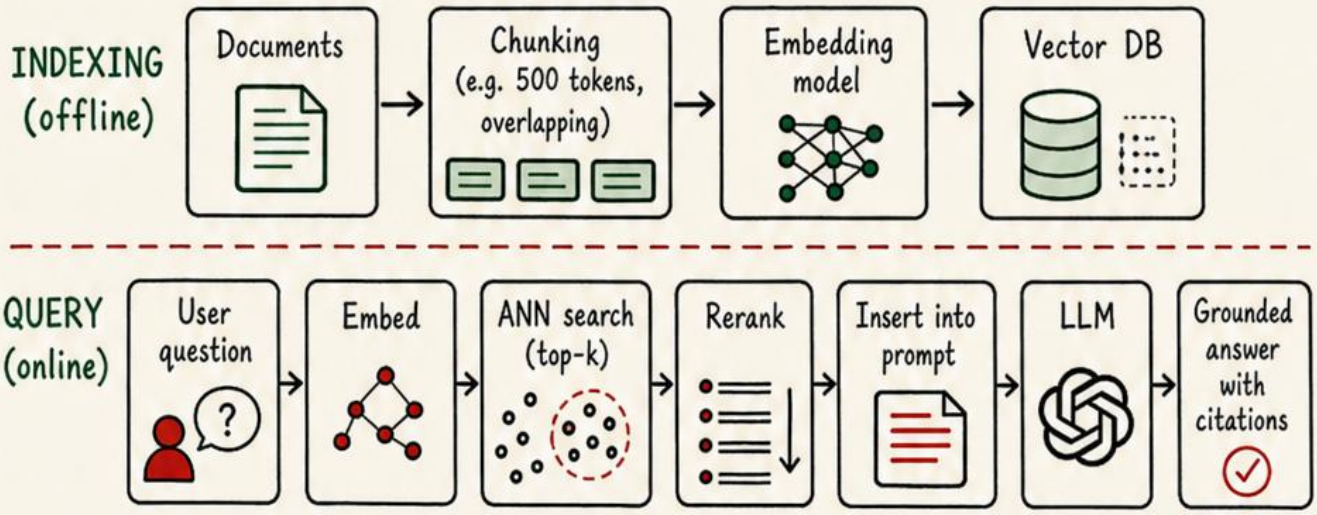


★ Context is a budget. Spend it on what changes the answer. ★

11 — RAG & TOOL CALLING

Give the model facts it never memorised, and hands it never had.

RETRIEVAL AUGMENTED GENERATION



VECTOR SEARCH

- Embeddings map text to vectors; similar meaning → nearby vectors
- ANN indexes: HNSW (graph, fast, memory hungry), IVF-PQ (compressed, cheaper)
- Cosine similarity is the usual metric
- Hybrid search = keyword (BM25) + vector, then rerank — beats either alone

Re-embed everything when you change the embedding model.

CHUNKING MATTERS

Too small → context is lost.

Too large → noise floods the prompt.

Use overlap, respect headings and paragraphs, and store metadata (source, section, date) for filtering and citations.

TOOL / FUNCTION CALLING FLOW

1. Send tool schemas along with the prompt
2. Model replies with a structured call: `search_flights(from, to, date)`
3. Backend VALIDATES the arguments
4. Backend executes the tool (never the model)
5. Result is appended as a tool message
6. Model is called again to produce the final answer

Repeat steps 2–6

Cap the number of tool iterations — loops are a real failure mode.

TOOL SAFETY

- ✓ Treat every model output as untrusted input
- ✓ Whitelist tools per user and per tier
- ✓ Enforce timeouts and quotas per tool
- ✓ Never let the model construct raw SQL or shell commands
- ✓ Log every call for audit and replay

WHEN NOT TO RETRIEVE

Skip retrieval for chit-chat, code formatting, or pure reasoning — it adds latency and tokens for nothing.
Use a cheap classifier to decide.

Retrieve to ground the answer. Call tools to change the world. ★

13 — CACHING LAYERS

Every cache hit is a GPU-second you did not pay for.

FOUR CACHES, FOUR PURPOSES

1	Exact response cache	identical prompt → stored answer	Hit rate: Very Low
2	Semantic cache	similar meaning → reuse answer	Hit rate: Low–Medium
3	Prefix / KV cache	shared prompt prefix → skip prefill	Hit rate: High
4	Retrieval cache	repeated queries → stored documents	Hit rate: Medium–High

1. EXACT CACHE

Key = hash(model + full prompt + parameters)

✓ Trivial, perfectly safe

✗ Almost never hits in open-ended chat

Best for: FAQs, system tasks, evaluation runs, temperature = 0

2. SEMANTIC CACHE

Embed the query; if similarity to a cached entry > threshold, reuse the answer.

✓ Real hit rates on support-style workloads

✗ Dangerous — "What is my balance?" must never hit another user's entry

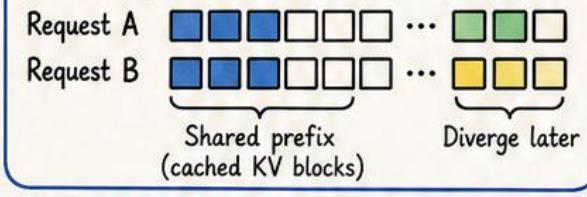
Rules:

- scope the cache per user or tenant
- set a high threshold
- never cache personalised or time-sensitive answers

3. PREFIX / KV CACHE (biggest win)

The system prompt and earlier turns repeat every request.
Reuse their KV blocks → prefill cost drops sharply → TTFT improves.

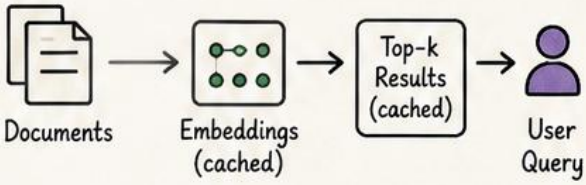
- Requires an exact token-level prefix match
- Route follow-ups to the same replica to hit it
- Evict by LRU when GPU memory is tight



4. RETRIEVAL CACHE

Cache embeddings and top-k results for repeated queries.

Embeddings for static documents are computed once, ever.



WHAT NOT TO CACHE

✗ Anything personalised across users
✗ Live data (prices, availability, news)
✗ High-temperature creative output
✗ Anything where staleness is a safety issue

Prefix caching helps latency.
Semantic caching helps cost.

Only one of them can be wrong about correctness.

★ Cache the prompt, not the person. ★

13 — RATE LIMITS, QUOTAS & COST

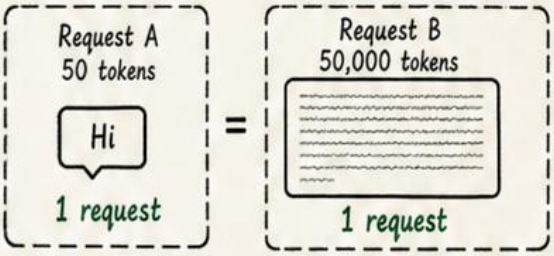
Requests per second is the wrong limit. Tokens per minute is the right one.

WHY TOKEN-BASED LIMITS

- One request can be 50 tokens or 50,000.
- Limiting requests lets a single user monopolise a GPU.

Limit on:

- TPM — tokens per minute (input + output)
- RPM — requests per minute (abuse guard)
- Concurrent generations per user
- Daily token quota per plan



TOKEN BUCKET, SIZED IN TOKENS

- Bucket refills at the plan's TPM rate
- Reserve max_tokens up front, refund the unused amount at the end
- Reject with 429 and a Retry-After header
- Counters in Redis, keyed per user and per organisation

⚠ Reserve before generating — otherwise a user can start a hundred long generations at once.

COST MODEL

Cost ≈ (input_tokens × prefill rate) + (output_tokens × decode rate)

Output tokens usually cost several times more than input tokens.

Levers that actually move the bill:

- Higher batch size → better GPU utilisation
- Prefix cache hits → less prefill
- Smaller model where quality allows
- Shorter system prompts
- Cap max_tokens per tier

METERING PIPELINE



- Meter at generation time, including cancelled and partial streams
- Idempotent by generation_id — never double-bill
- Show users their own consumption

ABUSE & FAIRNESS

- Free tier gets lower priority, not just lower limits
- Detect scraping and automated fan-out
- Per-organisation caps stop one team draining a shared quota
- Anomaly alerts on sudden token spikes

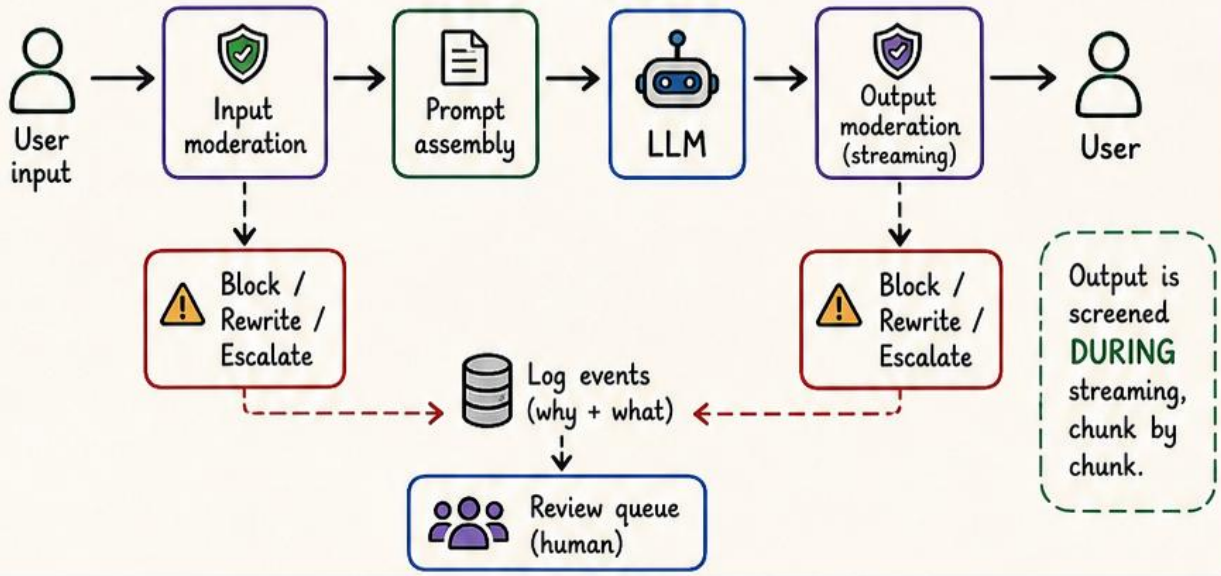
In LLM products, capacity planning and billing are the same conversation.

Meter tokens, limit tokens, optimise tokens.

14 — SAFETY, MODERATION & PRIVACY

Safety is a pipeline stage on both sides of the model, not a prompt instruction.

SAFETY PIPELINE



INPUT SIDE

- Classify for disallowed categories
- Detect prompt-injection attempts in pasted text and retrieved documents
- Strip or flag secrets and personal data
- Enforce per-user policy and age restrictions

OUTPUT SIDE

- Screen streamed chunks incrementally, not only at the end
- Stop the stream and replace the message if a violation appears
- Watch for leaked system prompts and training-data regurgitation
- Add citations where the answer claims facts

⚠ A streamed token cannot be un-sent — buffer a small window before displaying.

PROMPT INJECTION (the hard one)

Retrieved documents and tool results are **UNTRUSTED** text that reaches the model.

"Ignore previous instructions..." hidden inside a web page is an attack.

Defences:

- Keep instructions and data clearly separated
- Never grant tools authority based on model output alone
- Require confirmation for destructive actions
- Constrain tool arguments with strict schemas

There is no complete fix — reduce blast radius instead.

PRIVACY & DATA HANDLING

- Encrypt conversations at rest and in transit
- Clear retention windows and user-initiated deletion
- Opt-in, not opt-out, for training on user data
- Redact secrets before logging prompts
- Tenant isolation for enterprise customers
- Regional data residency where required

ABUSE CONTROL

- Rate limits
- Account reputation
- Automated fan-out detection
- Human review queue for repeat offenders

★ Assume every input is hostile and every output is public.

Screen both directions. Log carefully. Delete honestly. ★

15 — FAILURES & RELIABILITY

> A half-written answer is worse than no answer — design the failure modes deliberately.

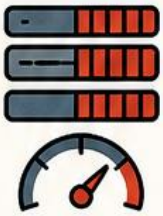
FAILURE MATRIX

1. GPU worker crashes mid-decode



replay the prompt on another worker; the request is idempotent, tokens already streamed are discarded or resumed

2. Queue depth explodes



shed load, shrink max_tokens, route to a smaller model

3. KV cache exhausted



preempt the lowest-priority sequence and requeue it

4. Client disconnects



cancel immediately, free the cache, bill partial usage

5. Vector DB or tool down



answer without retrieval and say so, rather than failing

6. Region outage



drain traffic to another region; warm replicas make this survivable

RESILIENCE PATTERNS

1. **Timeouts on every stage** (prefill, decode, tools)



2. **Circuit breakers around tools and retrieval**



3. **Bounded retries with jitter** — retries are expensive here



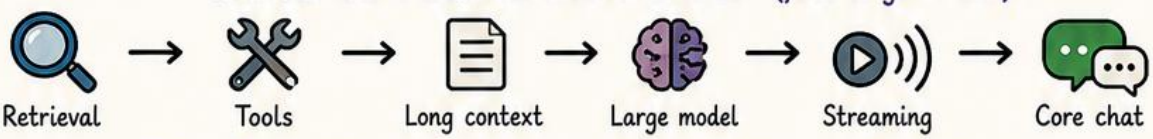
4. **Bulkheads:** separate pools per tier so free users cannot starve paid ones



5. **Idempotency key** prevents duplicate paid generations



GRACEFUL DEGRADATION ORDER (first to go → last)



Note: The chat box must always respond, even if the answer is smaller.

OBSERVABILITY — WHAT TO MEASURE

- ✅ TTFT p50 / p95 / p99
- ✅ TPOT and its variance (jitter is very visible to users)
- ✅ Queue wait time and depth
- ✅ Tokens/sec per GPU and GPU utilisation
- ✅ KV cache occupancy and preemption rate
- ✅ Prefix cache hit rate
- ✅ Error and cancellation rates
- ✅ Cost per 1K tokens by model and tier



QUALITY IS ALSO RELIABILITY

- ✅ Golden prompt set run on every deploy
- ✅ Regression evaluations before model or prompt changes
- ✅ Canary a new model version on a small traffic slice
- ✅ Track thumbs-up/down and regeneration rate as live signals

A model upgrade can be a production incident — treat prompts and weights as versioned artefacts.



Degrade the answer, keep the conversation alive. ★

16 — OPTIMIZATION TOOLKIT

Six levers. Know what each one costs you in quality.

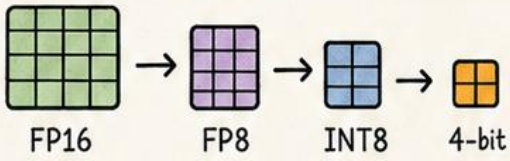
1. QUANTIZATION

Store weights in fewer bits:

FP16 → FP8 → INT8 → 4-bit

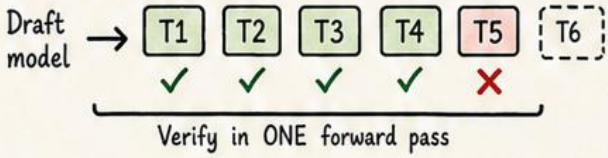
✓ Less memory, faster reads, bigger batches

✗ Small quality loss; needs calibration



2. SPECULATIVE DECODING

A small draft model proposes k tokens ahead. The large model verifies them in ONE forward pass. Accepted tokens are kept; the first rejection restarts from there.



✓ Often 2–3× faster decoding with identical output distribution

✗ Extra memory for the draft model; gains depend on acceptance rate

3. CONTINUOUS BATCHING

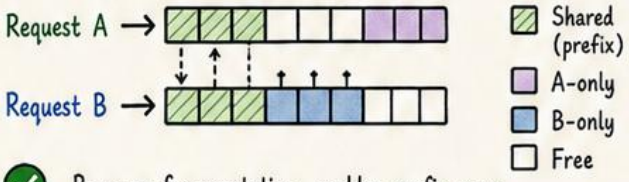
Covered on page 9 — the single biggest throughput lever.

✓ 2–4× throughput

✗ Needs careful admission control

4. PAGED KV CACHE

Block-based cache with sharing of common prefixes.

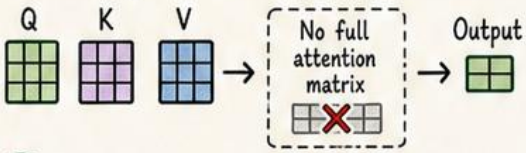


✓ Removes fragmentation, enables prefix reuse

✗ Slight indirection overhead

5. FLASH ATTENTION / FUSED KERNELS

Memory-efficient attention that avoids materialising the full attention matrix.

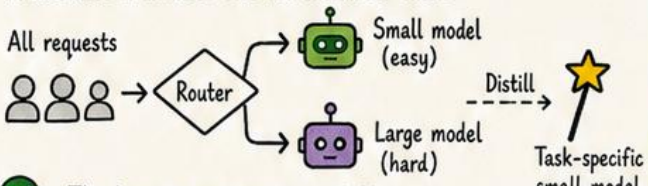


✓ Faster and far less memory for long contexts

✗ Hardware and library specific

6. MODEL RIGHT-SIZING & DISTILLATION

Route easy requests to a small model; distil a big model into a smaller one for a narrow task.



✓ The largest cost saving available

✗ Requires evaluation to prove quality holds

PICK BY BOTTLENECK

TTFT too high →
- prefix cache
- chunked prefill
- more replicas

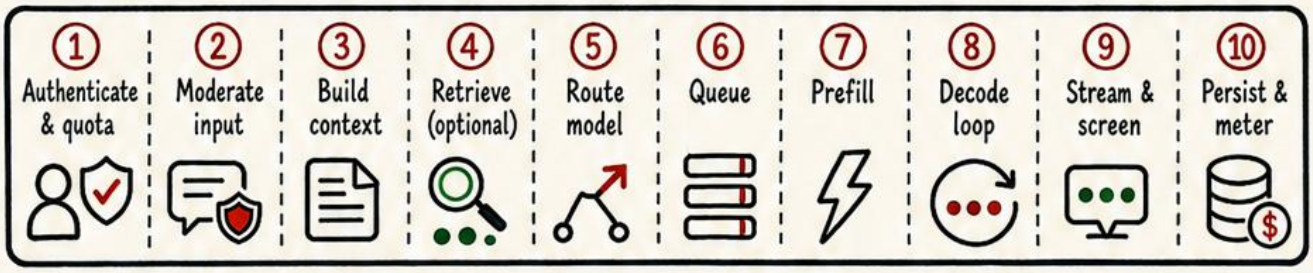
TPOT too high →
- quantization
- speculative decoding
- smaller model

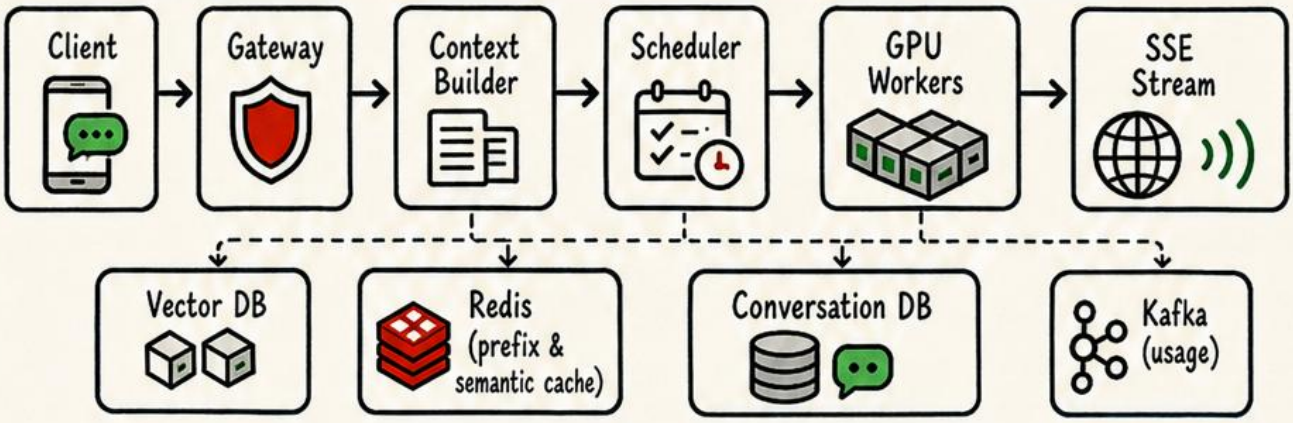
Throughput too low →
- continuous batching
- larger batch
- paged cache

Cost too high →
- routing
- right-sizing
- shorter prompts

Profile first. Every optimisation trades something away. ★

17 — RECAP & INTERVIEW CHEAT SHEET





DESIGN DECISIONS

- SSE for token streaming; REST for conversation management
- Continuous batching with iteration-level scheduling
- Paged KV cache with prefix sharing
- Sticky routing by conversation to hit the prefix cache
- Token-based rate limits, not request-based
- Moderation on both input and output streams
- Cancel on disconnect and free the cache immediately
- Degrade to a smaller model instead of failing
- Meter every generation, including partial ones

INTERVIEW FLOW — 7 STEPS

1. Clarify scope
2. Estimate tokens and GPUs
3. APIs & data model
4. Draw architecture
5. Deep dive the scheduler
6. Failures & cost
7. Trade-offs

MOST-ASKED QUESTIONS

- How do you stream tokens to millions of users?
- What is TTFT and how do you reduce it?
- Prefill vs decode — why do they scale differently?
- What limits how many users share one GPU?
- How does continuous batching improve throughput?
- How do you rate limit fairly when requests vary hugely?
- How do you handle a client that disconnects mid-generation?
- How would you cut inference cost by half?
- How do you defend against prompt injection?

30-SECOND SUMMARY

The web tier is an ordinary stateless chat application; the difficulty is GPU scheduling. A request is authenticated and moderated, context is assembled within a token budget, and the job enters a priority queue. A worker runs a compute-bound prefill, then a memory-bound decode loop that emits one token at a time over SSE. Continuous batching keeps every GPU slot full, a paged KV cache lets sequences share prefixes, and everything is metered in tokens because tokens are the unit of both latency and cost.



Fast First Token. • Full Batches. • Safe Output.

HANDBOOK COMPLETE ✓